



CSE422 Lab Project Report
Spring'26

Heart Disease

Sec 11

Group 02

Fabiha Fairose (ID: 23201244)

Shazneen Ahmed Nabila (ID: 23201342)

Table of Contents

1. Introduction	02
2. Dataset Description	03 – 20
3. Dataset Pre-processing	20– 22
4. Dataset Splitting	22– 23
5. Model Training & Testing	23 – 33
6. Model Comparison	33 – 36
7. Conclusion	36-37

1.Introduction

The objective of this project is to apply machine learning techniques to predict the presence of heart disease and severity of it based on patient medical data. Heart disease is one of the leading causes of death worldwide, and early detection can significantly improve treatment outcomes.

This project follows a complete machine learning pipeline, starting from data exploration to model evaluation. Multiple supervised learning models are applied and compared to determine the most effective approach for classification. Additionally, an unsupervised learning method (K-Means clustering) is used to explore hidden patterns in the dataset.

The motivation behind this project is to understand how machine learning can assist in healthcare decision making by identifying patterns in clinical data.

2.1 Dataset Description

- The dataset contains 15 features related to patient health conditions, including age, dataset, sex, cp (chest pain type), trestbps (resting blood pressure), chol (cholesterol level), fbs (fasting blood sugar), restecg (resting ECG results), thalach (maximum heart rate achieved), exang (exercise-induced angina), oldpeak (ST depression), slope (slope of ST segment), ca (number of major vessels), thal (thalassemia) and a target feature(num) which indicates the absence or presence with severity of heart

disease, making it the output variable, while the remaining features are used as input variables for prediction.

- The problem is a **classification problem**, as the goal is to predict whether a patient has heart disease or not (with severity, making it a multi class classification problem). The output variable consists of discrete categories rather than continuous values, which makes it suitable for classification models.

- The dataset consists of 920 data points (patient records), where each row represents an individual and each column(total 15) represents a specific medical attribute.

- The dataset contains both quantitative (numerical) and categorical (qualitative) features:

Quantitative features(6):

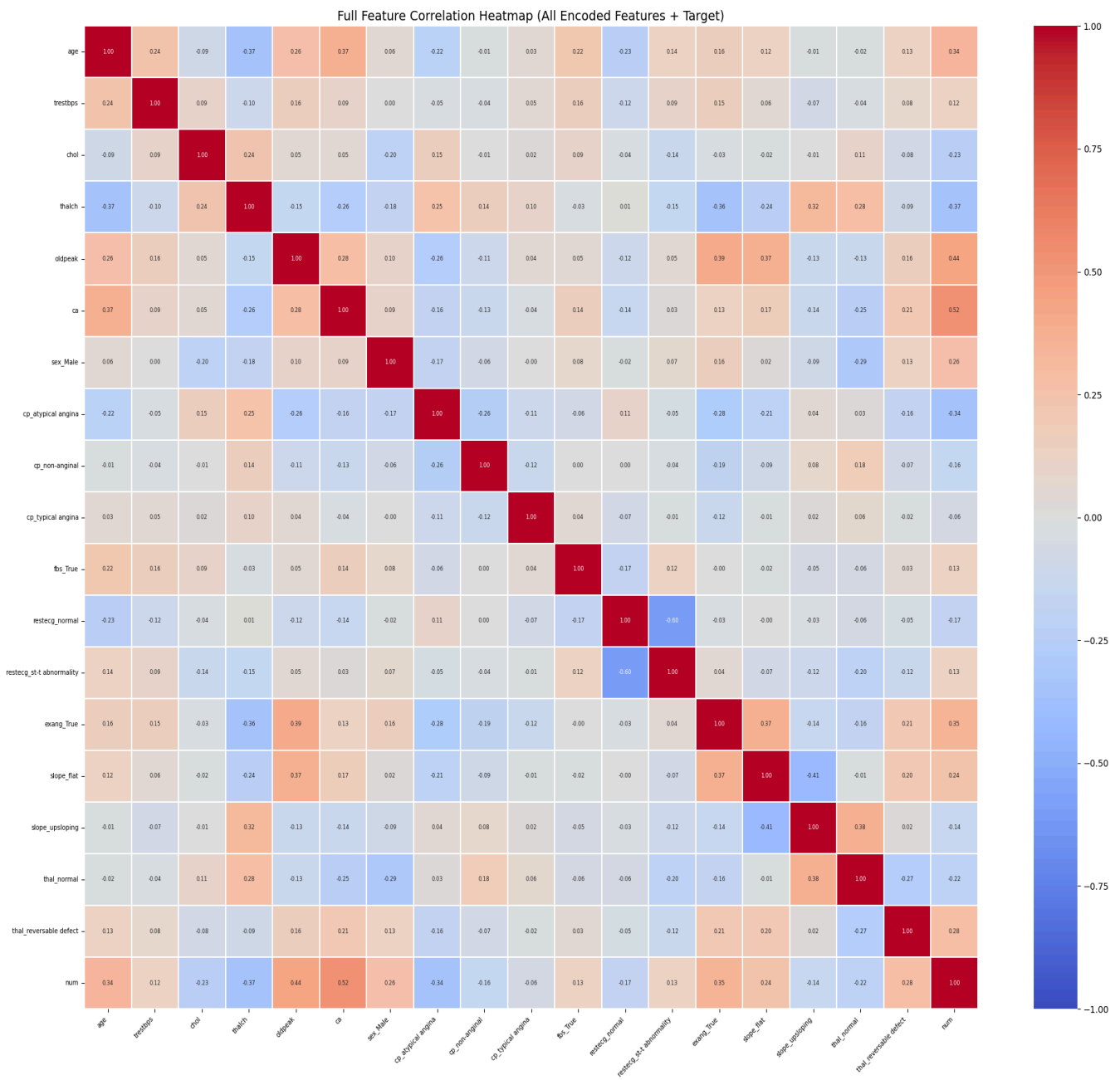
age, trestbps, chol, thalach, oldpeak, ca

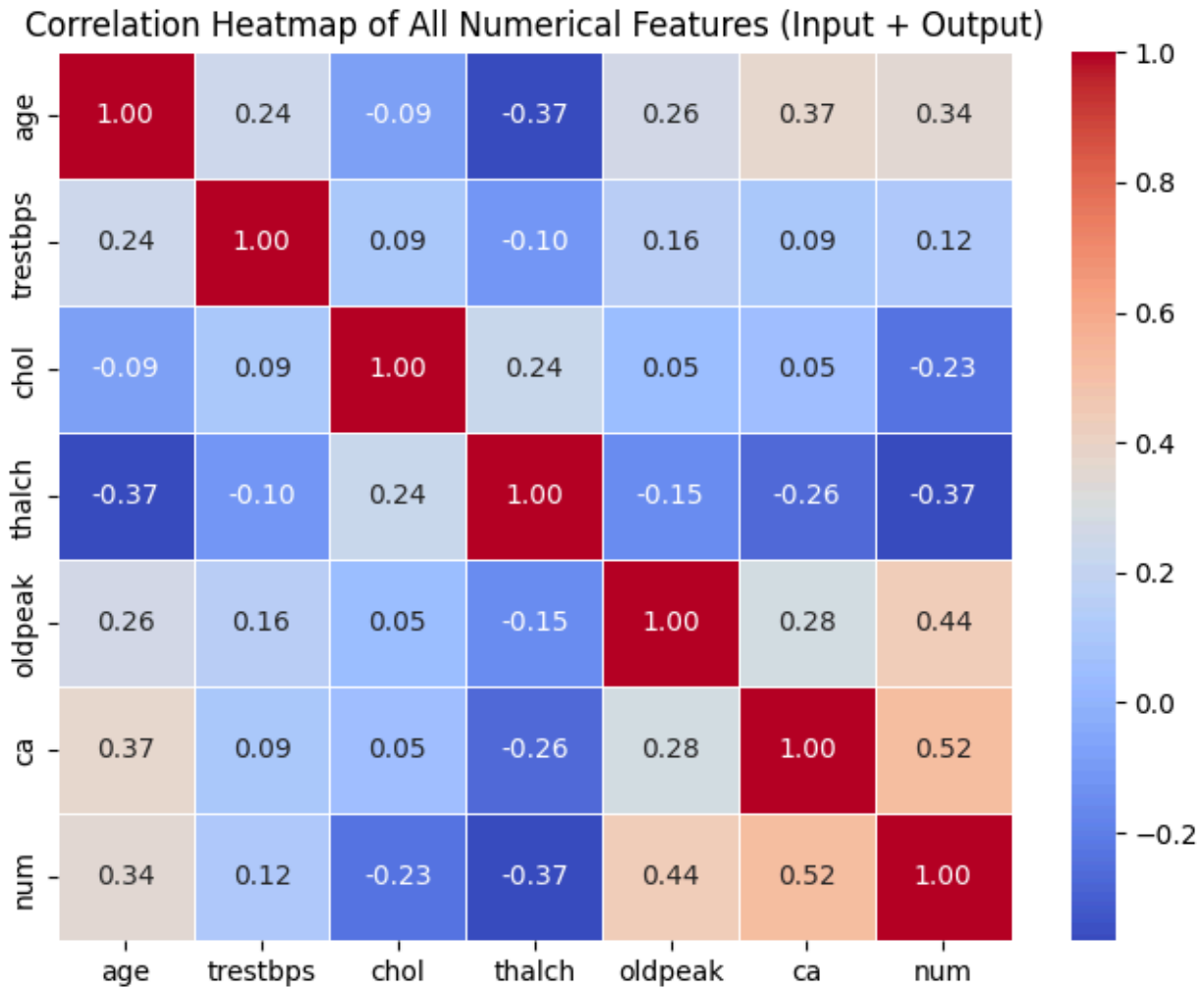
Categorical features(8):

sex, dataset, cp, fbs, restecg, exang, slope, thal

- Yes, we need to encode the categorical variables. Machine learning models require numerical input and categorical features such as chest pain type, ECG results, and exercise-induced angina cannot be processed directly. Therefore, **one-hot encoding** is applied to convert these features into numerical form. This method ensures that no false ordering is introduced among categories.

- A correlation heatmap was generated using the seaborn library to analyze the relationships between features.



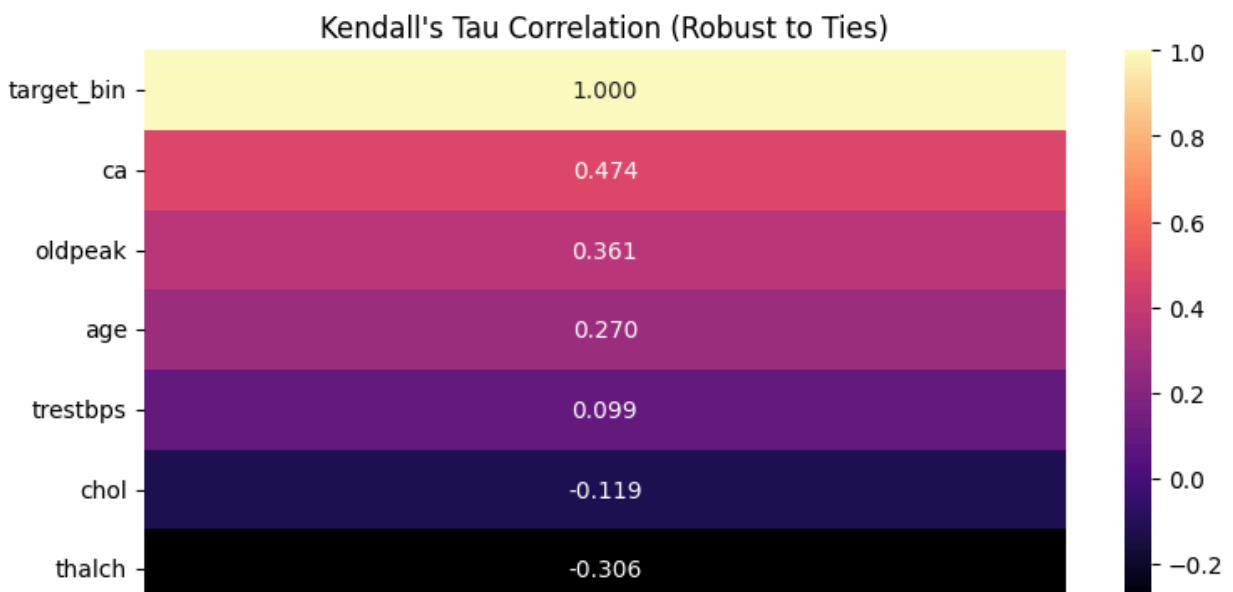
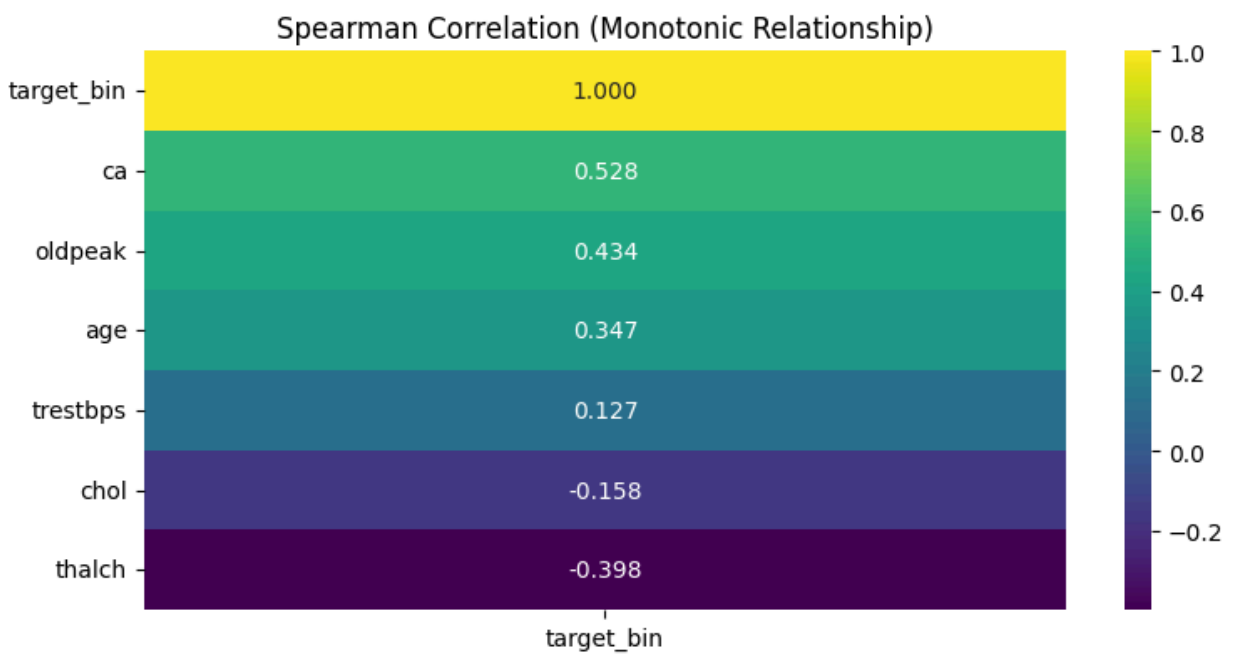
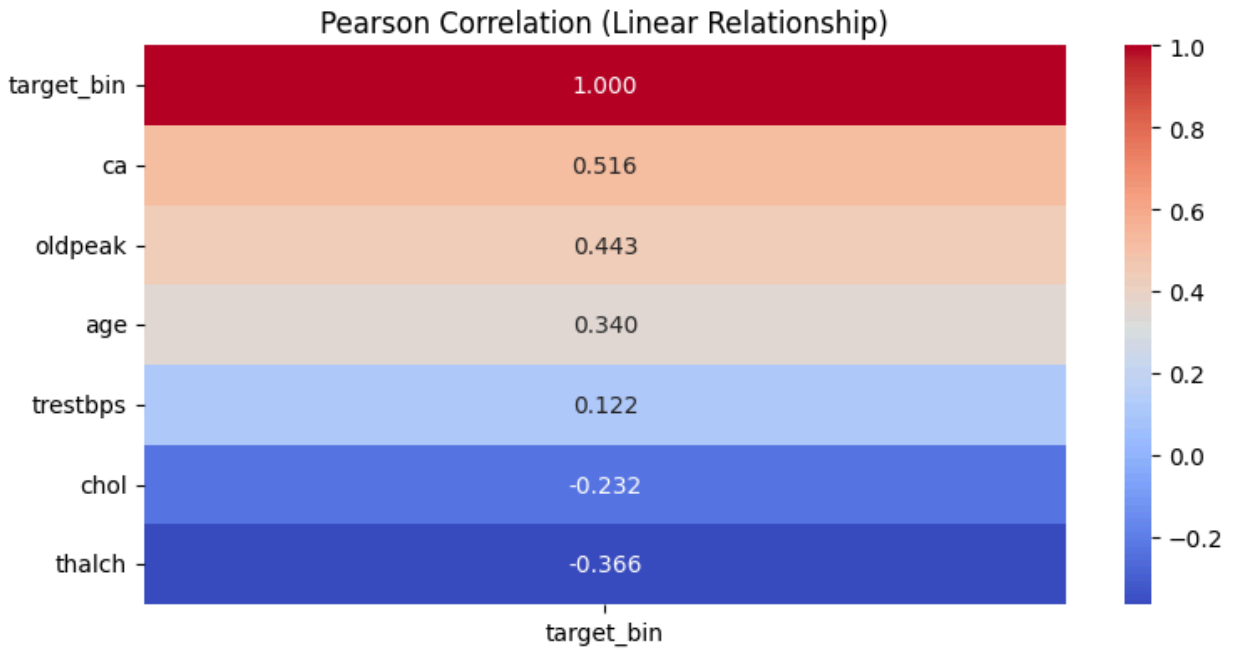


The correlation test measures how strongly two variables move together and in which direction (positive or negative). It helps you find relationships between features and detect highly related features (multicollinearity).

We understand from the Correlation Test

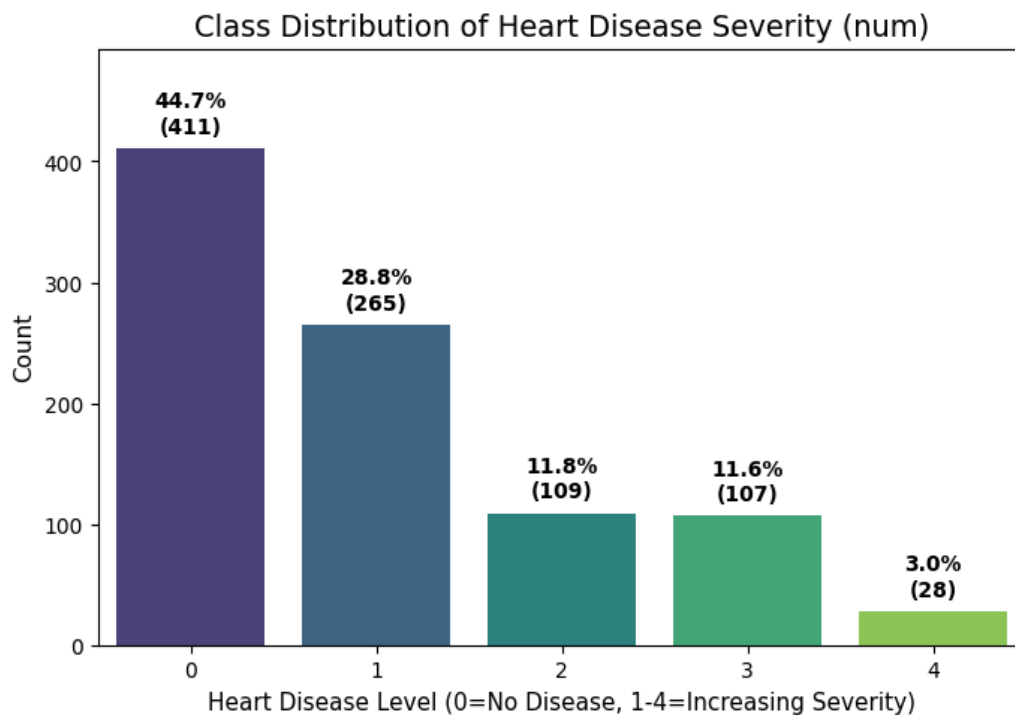
1. age and oldpeak show moderate positive correlation with the target variable, suggesting that older patients and those with higher ST depression tend to have more severe heart disease.
2. thalch (maximum heart rate) shows a negative correlation with the severity level, meaning that patients achieving higher maximum heart rates during exercise tend to have less severe heart disease.
3. ca (number of blocked vessels) shows the strongest positive correlation with severity among the numerical features -more blocked vessels correspond to more severe disease.
4. chol (cholesterol) and trestbps (blood pressure) exhibit relatively weak direct correlation with the target, though they are medically significant risk factors.
5. oldpeak and ca are positively correlated with each other, reflecting that patients with more blocked vessels also tend to have higher ST depression under stress.

Overall, the correlation analysis confirms that ca and oldpeak are the most predictive numerical features, while thalch inversely relates to disease severity. Most features show low inter-feature correlation, suggesting they contribute independently and that multicollinearity is not a major concern for this dataset.



The correlation heatmaps across all three analyses consistently show that `ca` (blocked vessels) and `oldpeak` (ST depression) are the strongest positive predictors of heart disease severity, while `thalch` (maximum heart rate) shows the strongest negative correlation with the target. `age` and `trestbps` show weak to moderate positive correlations, whereas `chol` shows a slight negative correlation with severity. The full encoded feature heatmap further reveals that `exang_True` (exercise-induced angina) and `slope_flat` also positively correlate with higher severity classes. These findings are consistent across Pearson, Spearman, and Kendall's Tau methods, confirming that the relationships are genuine and not driven by outliers or non-linearity.

2.2 Imbalance



The dataset is imbalanced, as the classes of the target variable `num` do not have an equal number of instances.

The majority class '0' (no heart disease) contains approximately 411 instances (44.7%), while the other classes represent different levels of heart disease severity:

- Class '1' contains approximately 265 instances (28.8%)
- Class '2' contains approximately 109 instances (11.8%)
- Class '3' contains approximately 107 instances (11.6%)
- Class '4' contains approximately 28 instances (3.0%)

This shows that the dataset is multi-class imbalanced, where lower severity levels (especially class 0 and 1) dominate the dataset, while higher severity levels (particularly class 4) have significantly fewer samples.

2.3 Exploratory Data Analysis(EDA)

Data Overview

```

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 920 entries, 0 to 919
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         920 non-null    int64
 1   sex         920 non-null    object
 2   dataset     920 non-null    object
 3   cp          920 non-null    object
 4   trestbps    861 non-null    float64
 5   chol        890 non-null    float64
 6   fbs         830 non-null    object
 7   restecg     918 non-null    object
 8   thalch      865 non-null    float64
 9   exang       865 non-null    object
10   oldpeak     858 non-null    float64
11   slope       611 non-null    object
12   ca          309 non-null    float64
13   thal        434 non-null    object
14   num         920 non-null    int64
dtypes: float64(5), int64(2), object(8)
memory usage: 107.9+ KB

```

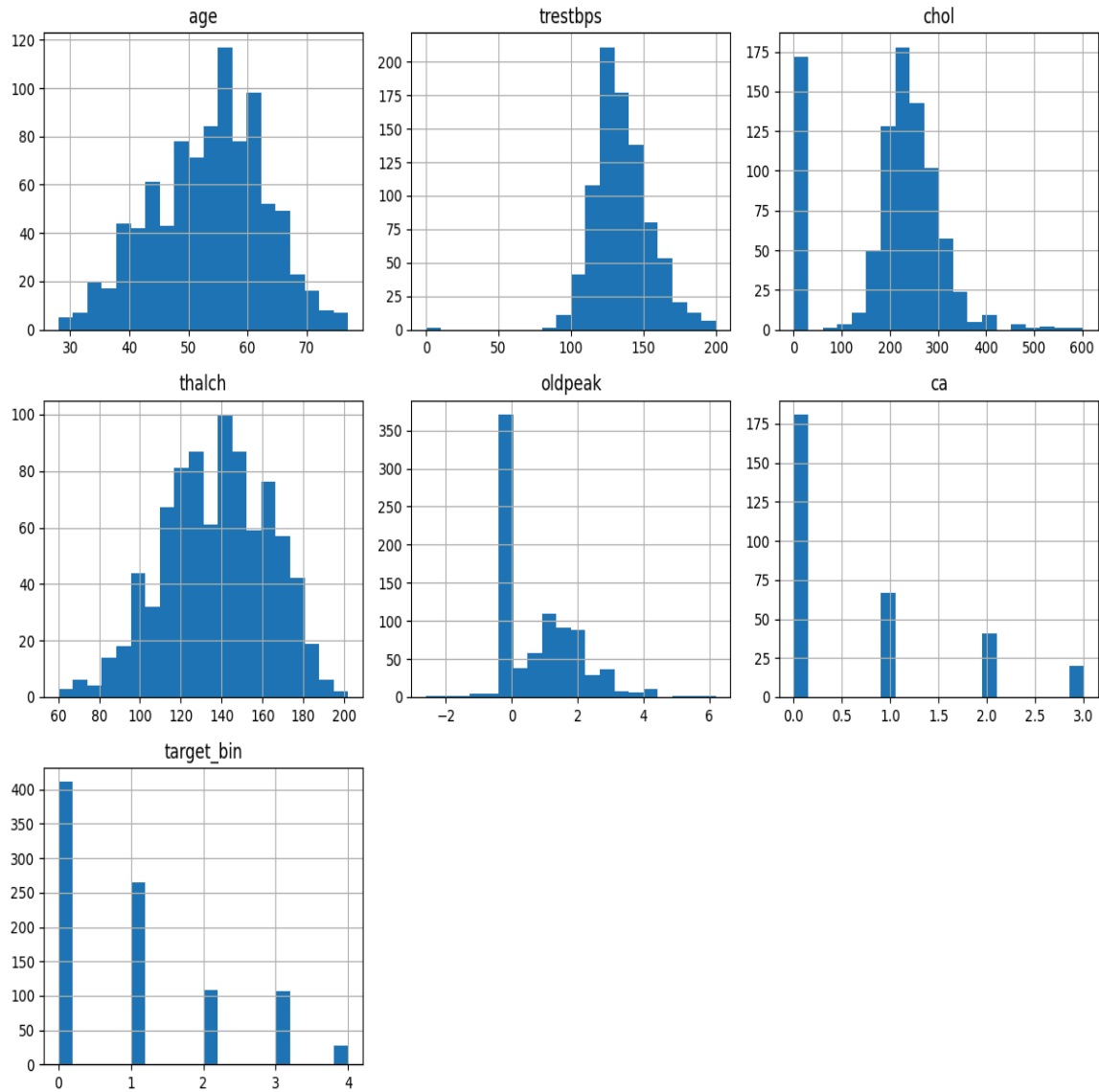
Exploratory Data Analysis was performed to understand the distributions, outliers, and relationships between features. The following visualizations were produced:

Skewness of Numerical Features:

```
Skewness of numerical features:  
age      -0.195994  
trestbps  0.213334  
chol     -0.613836  
thalch   -0.211119  
oldpeak  1.041427  
ca       1.165978  
target   0.968880  
dtype: float64
```

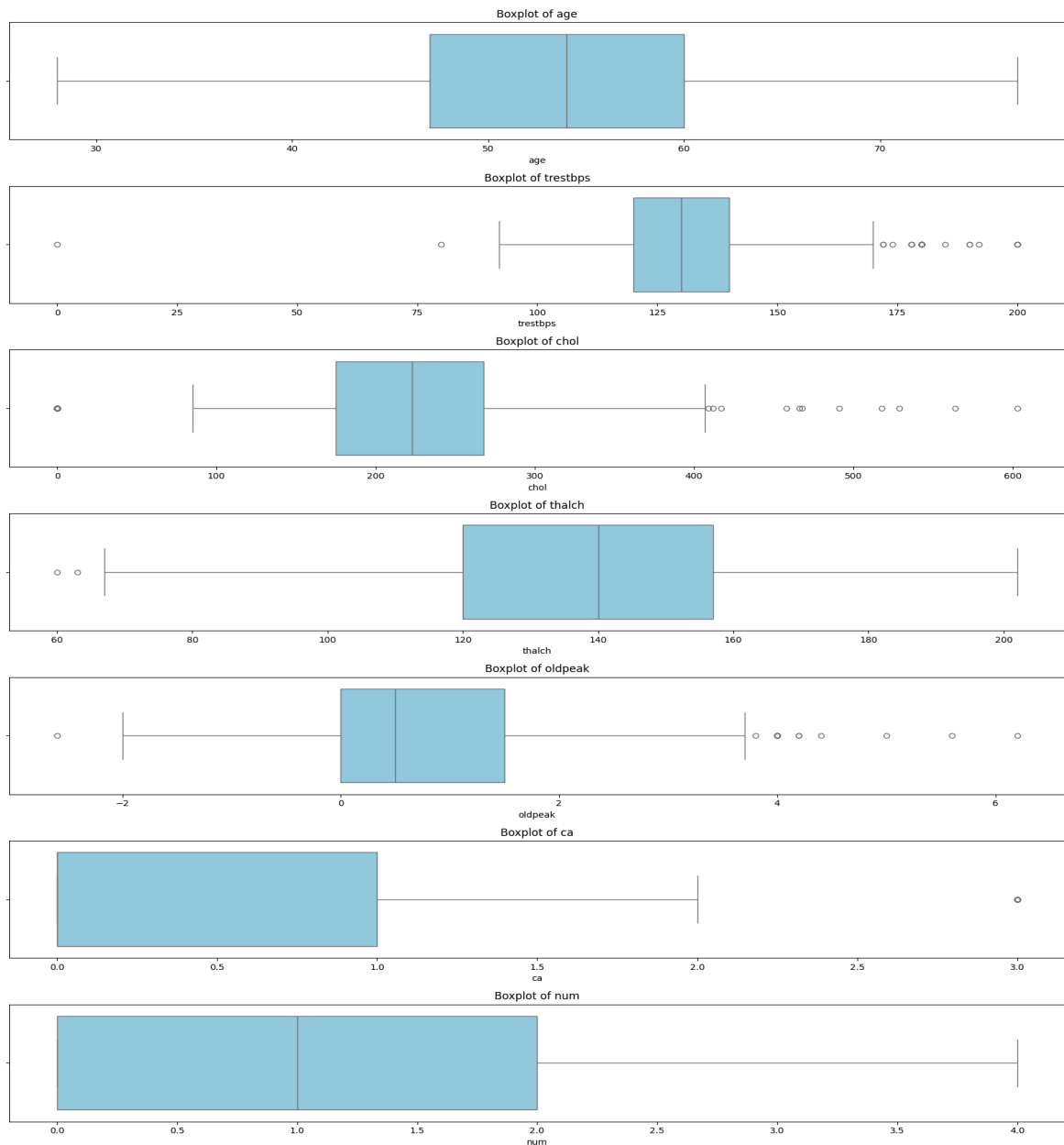
Histograms of Numerical Features:

Histograms of Numerical Features



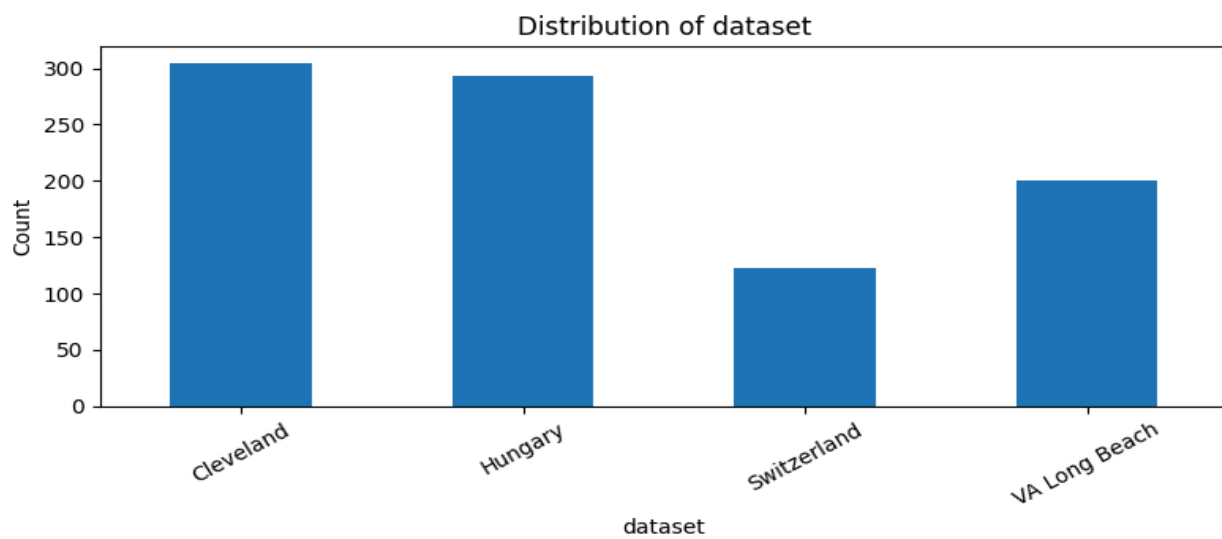
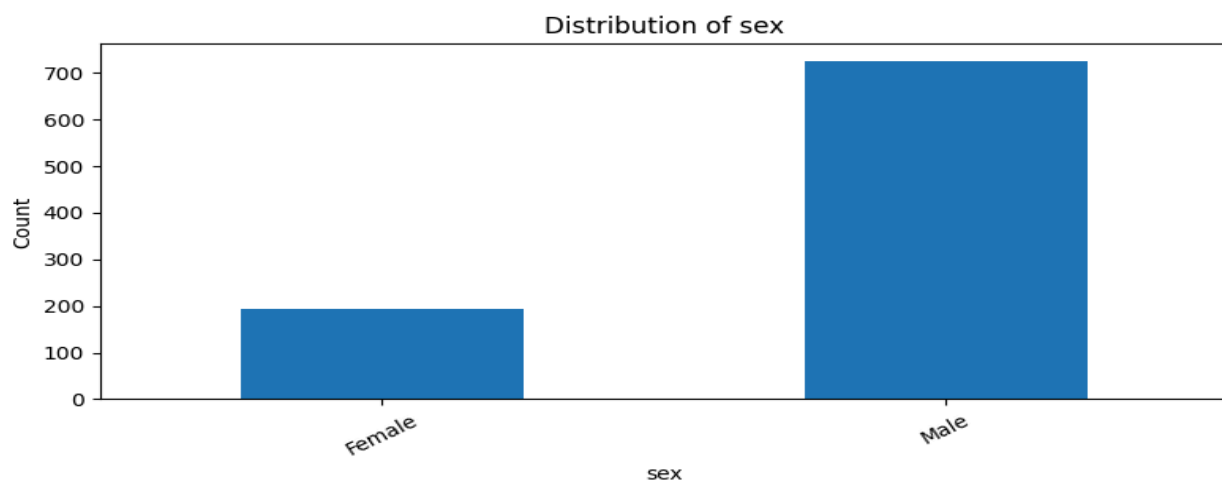
The histograms of age, trestbps, and thalch show approximately normal distributions, while oldpeak and ca are strongly right-skewed. oldpeak has a skewness of 1.04 and ca has a skewness of 1.17, indicating the presence of extreme high values. chol shows a slight left skew (-0.61), reflecting a long tail of very high cholesterol readings.

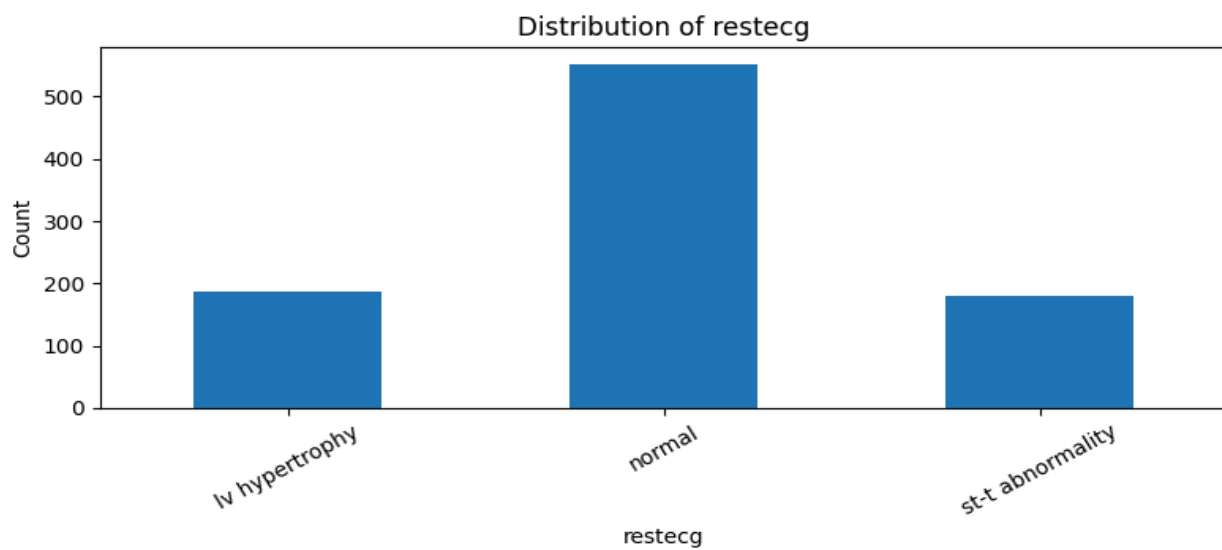
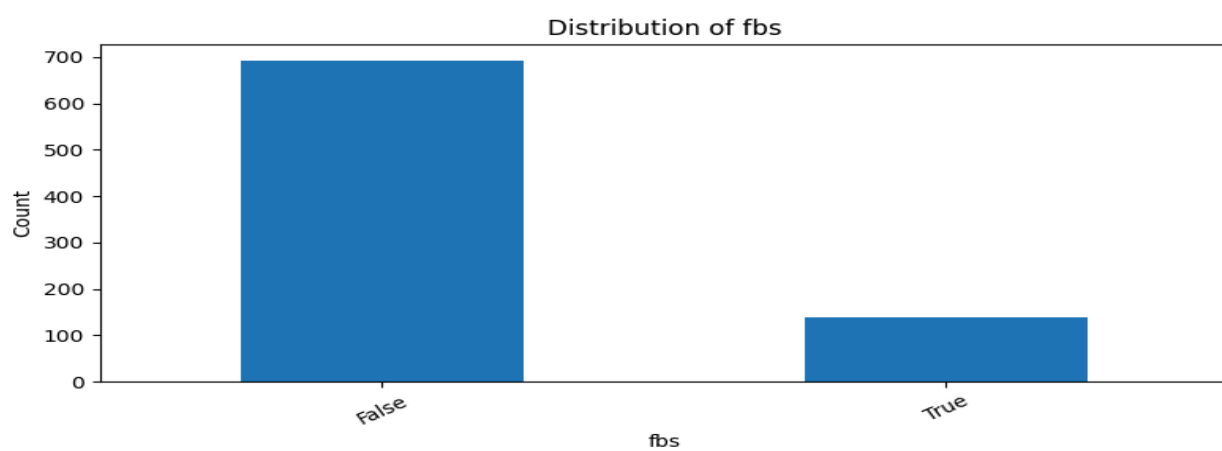
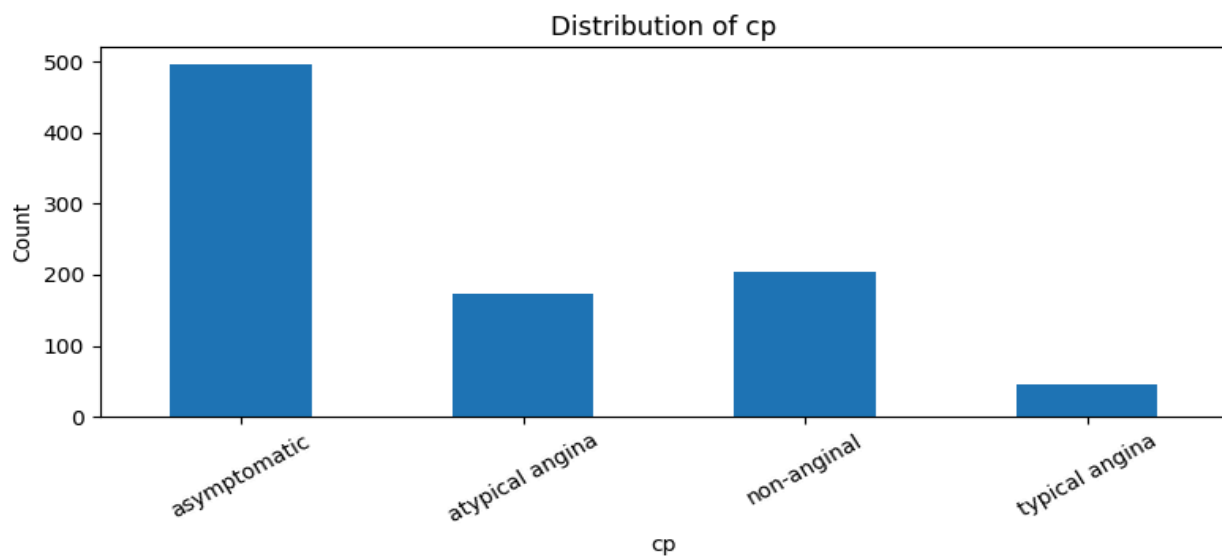
Boxplots and Outlier Detection:

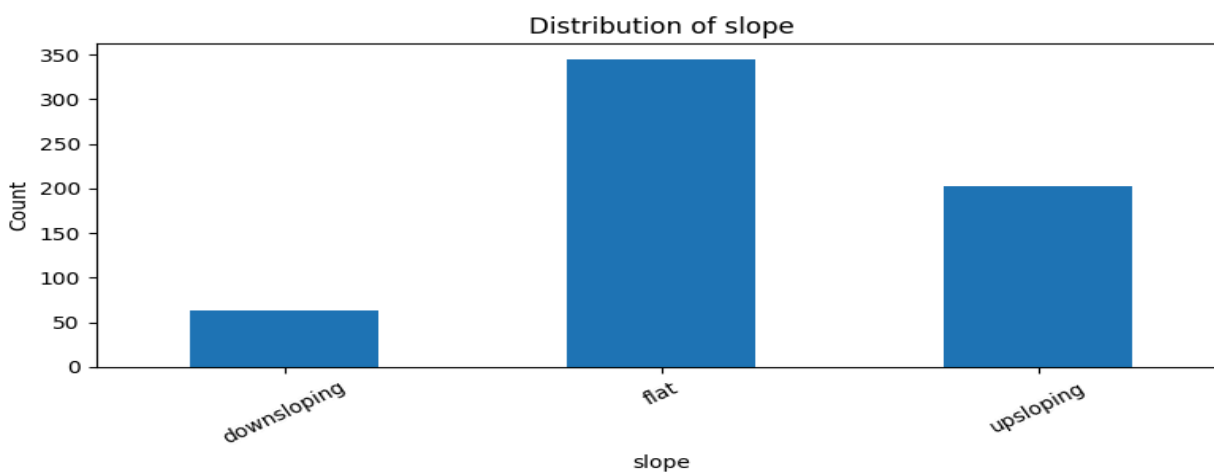
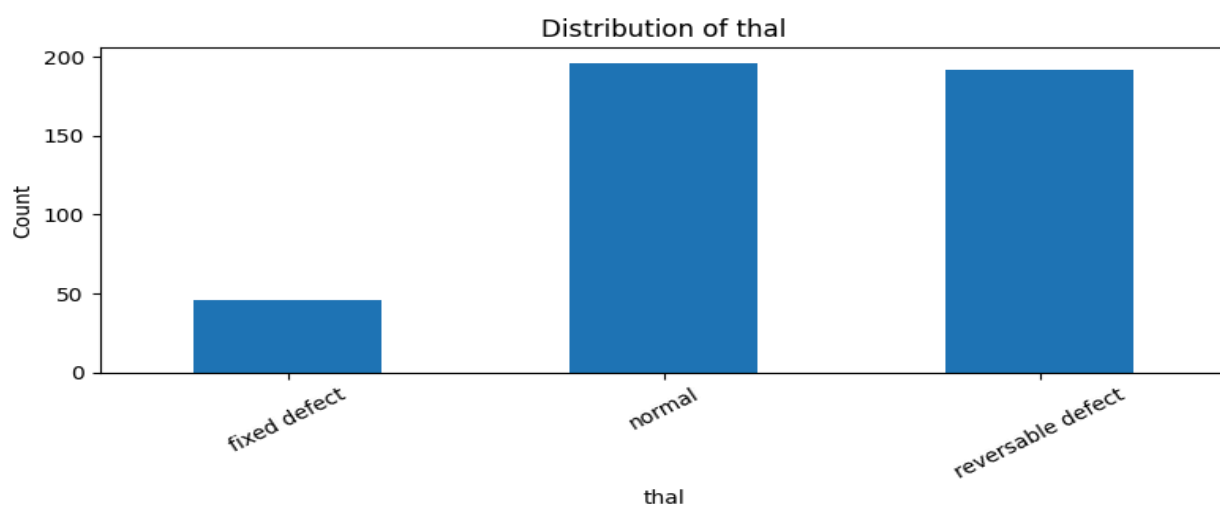
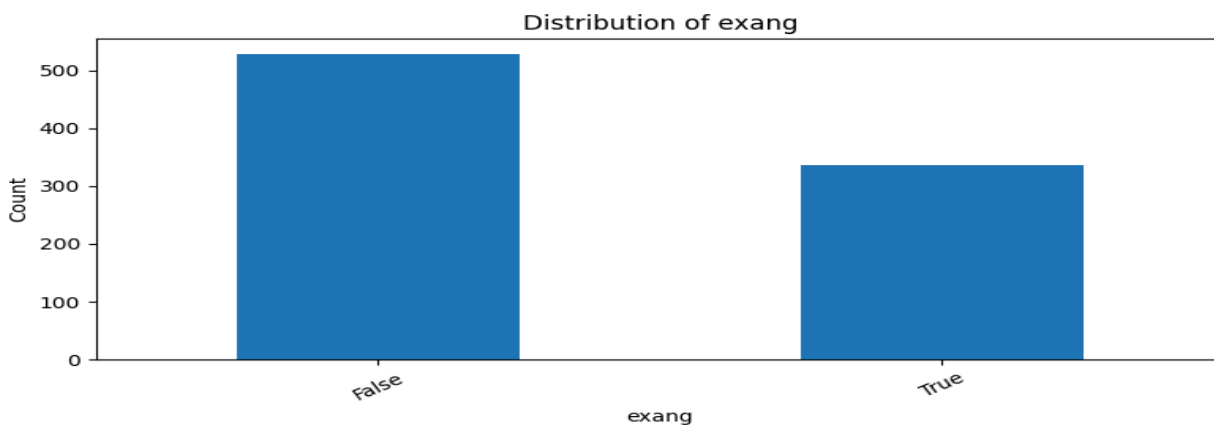


Boxplots revealed the presence of outliers in several numerical features. The trestbps feature contains a minimum value of 0.0, which is physiologically impossible and represents data entry errors rather than true measurements. chol also contains zero values, likely indicating missing data coded as 0. oldpeak ranges from -2.6 to 6.2 — while negative values are theoretically possible, extreme high values indicate outliers. These outliers are retained since imputation was applied for true missing values and StandardScaler provides robustness to extreme values.

Categorical Feature Distributions:

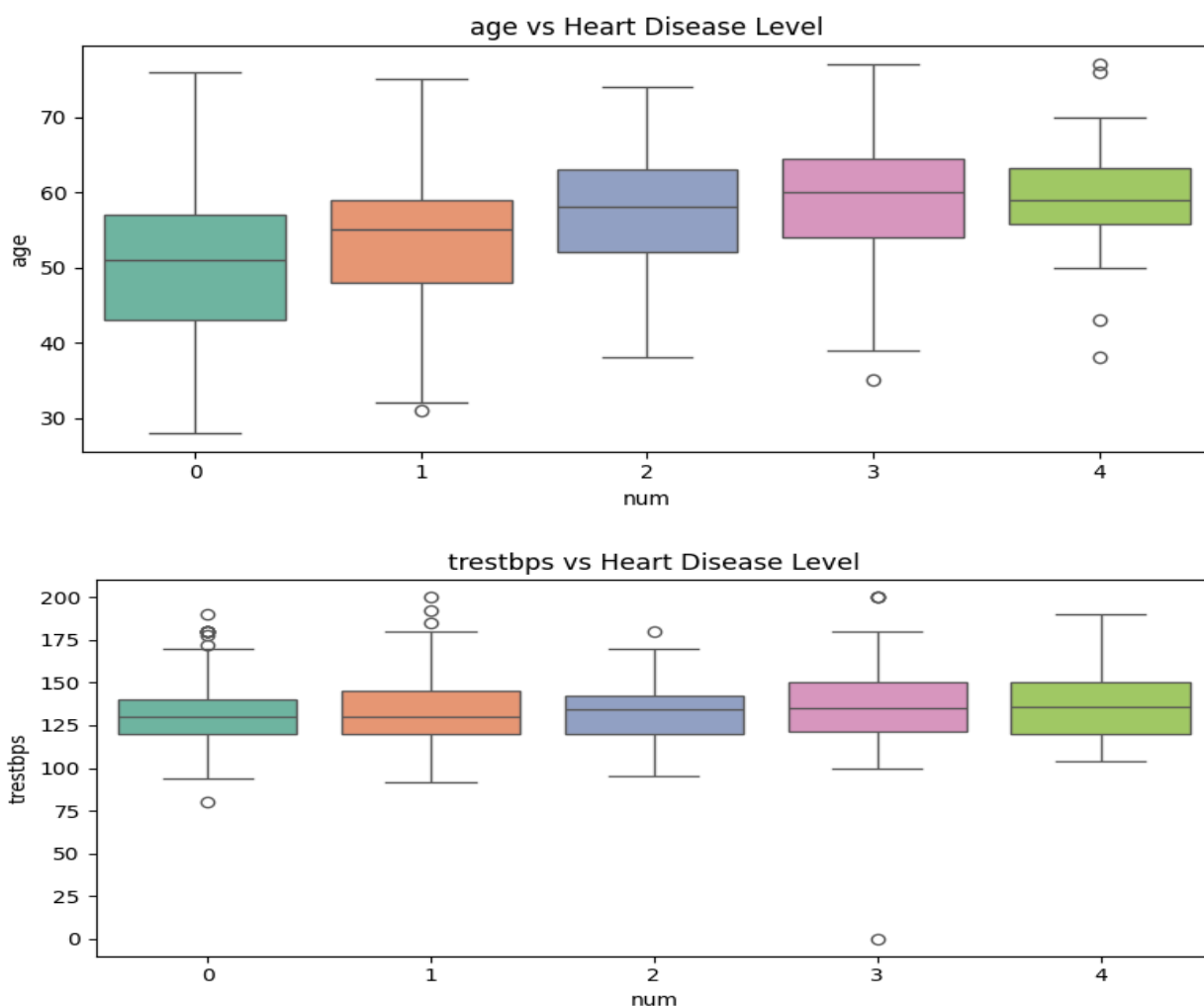


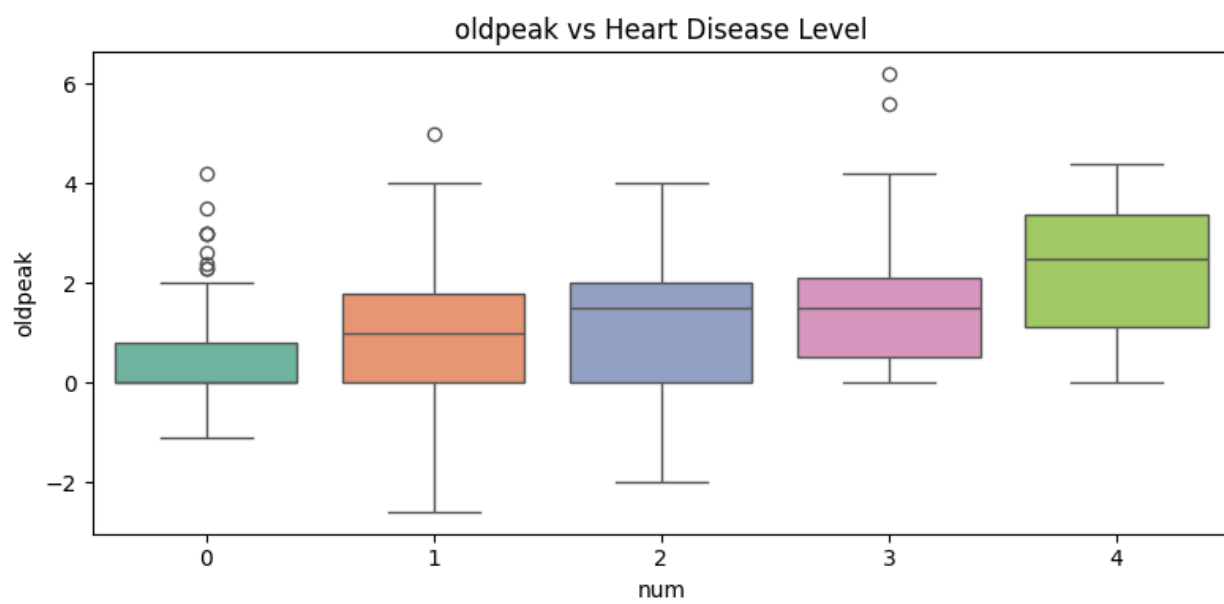
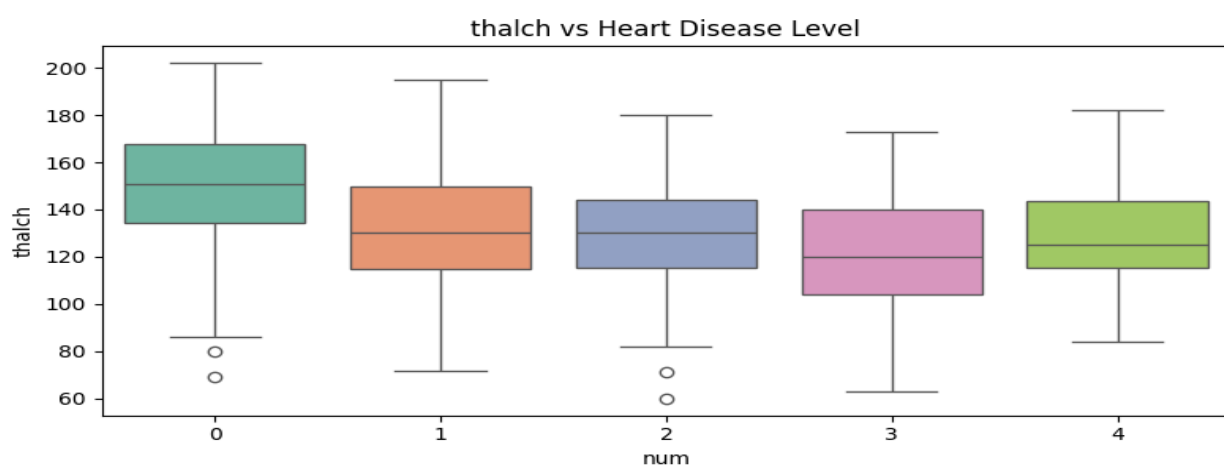
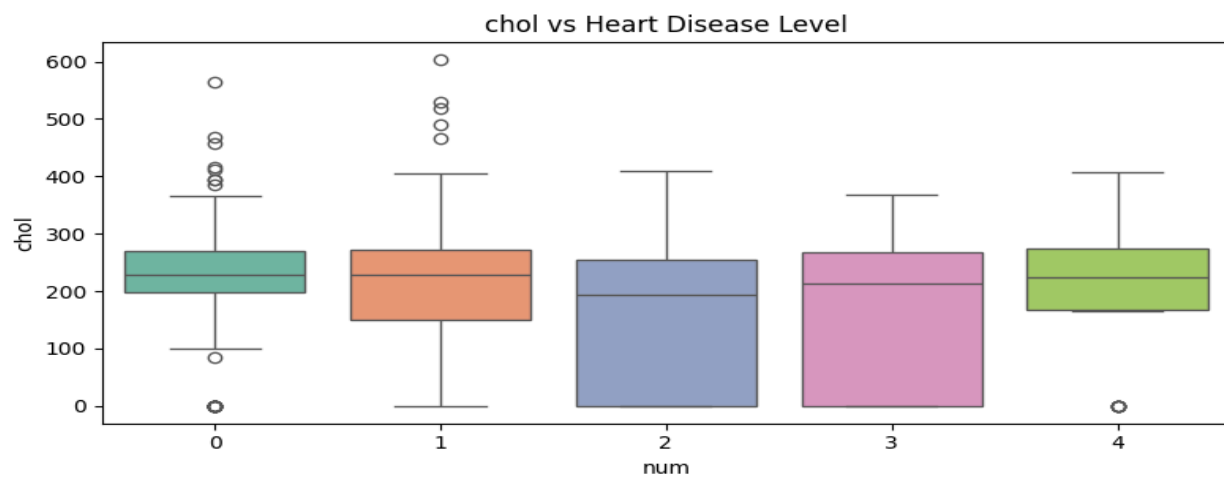




Bar charts of categorical features show that Male patients dominate the dataset (~79%). The most common chest pain type is asymptomatic, followed by non-anginal. The majority of patients have no exercise-induced angina (exang = False). The flat slope is the most common ST segment slope. These distributions provide useful context for interpreting model predictions.

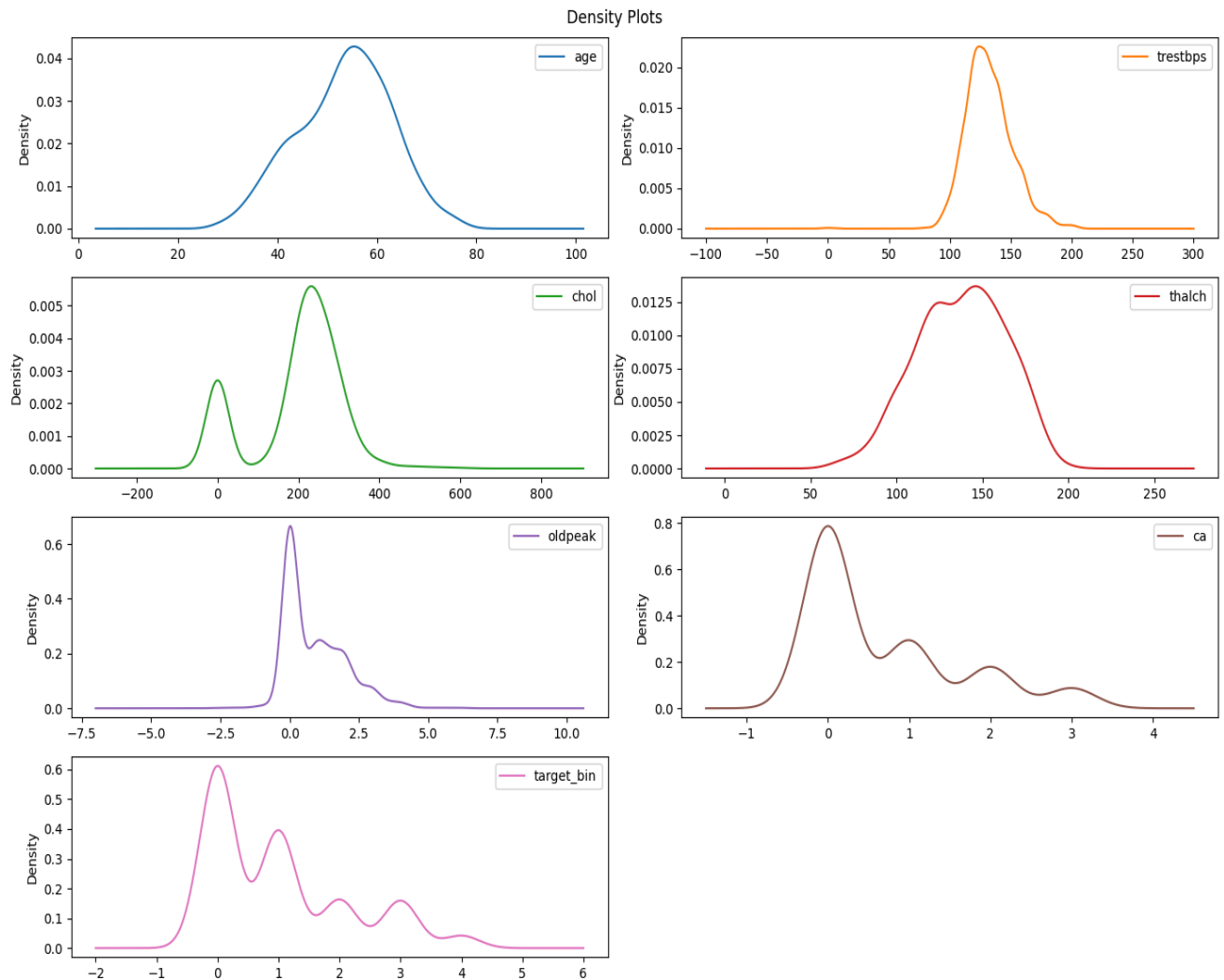
Feature vs. Target (Boxplots per Class):





Boxplots of each numerical feature split by the 5 heart disease severity classes reveal meaningful trends. age tends to increase slightly with severity. thalch shows a clear downward trend as severity increases. patients with no disease achieve higher maximum heart rates. oldpeak increases with severity, consistent with the correlation analysis. These trends confirm that the numerical features carry real predictive signal for the severity classes.

Density plots:



The density plots reveal the distribution of each numerical feature. The age feature follows a near-normal distribution centered around 45–55 years. trestbps (resting blood pressure) is approximately normally distributed around 120–130 mmHg, with a slight right tail indicating some patients with very high blood pressure. chol (cholesterol) shows a bimodal distribution with two peaks, suggesting two distinct patient sub-groups with different cholesterol profiles. thalch (maximum heart rate) shows a left-skewed distribution, with most patients achieving heart rates between 100–175 bpm. oldpeak is heavily right-skewed and spiked near 0, indicating that the majority of patients experienced little to no ST depression, with a long tail of extreme values representing a smaller group of high-risk patients. The target_bin density shows a multimodal distribution across its range (0–5), reflecting the five severity classes and confirming the imbalanced nature of the dataset.

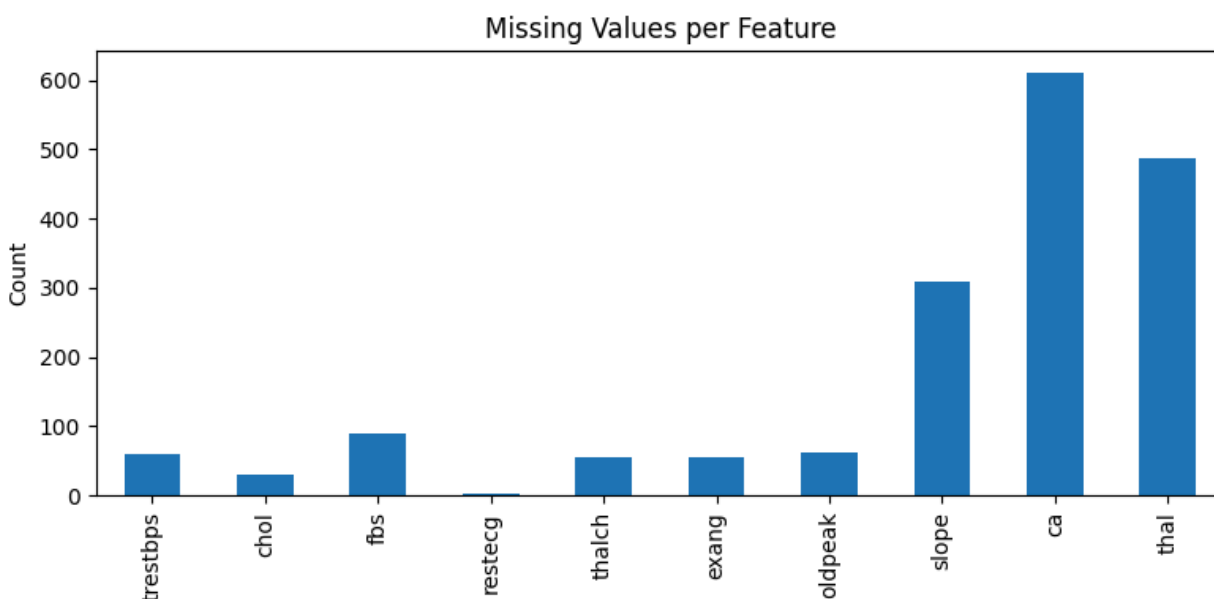
3. Dataset pre-processing

3.1 Fault 1: Missing values

Problem: The dataset had a lot of missing values. Features 'ca' had about 66% missing values and 'thal' had about 53% missing values. Other features like trestbps, chol also had some missing entries.

Solution: Column 'ca' were completely dropped because more than half the data was missing imputing would introduce too much noise. For other columns with fewer missing values, numerical features were filled with the median (because median is not affected by outliers), and categorical features were filled with the

mode (most frequent value). This is called imputation.



3.2 Fault 2: Categorical encoding

Problem: Features like sex, cp, restecg, exang, slope, and others are categorical and cannot be directly used by machine learning models.

Solution: One-Hot Encoding was applied using `pd.get_dummies()`. This creates separate binary columns for each category. For example, chest pain type (cp) with 4 values becomes 3 new columns. Binary features like sex and fbs were automatically handled as 0/1.

3.3 Fault 3: Feature scaling

Problem: Numerical features like chol (cholesterol, range ~100–600) and age (range ~20–80) have very different scales. Distance-based models like Neural Networks are sensitive to this.

Solution: StandardScaler was applied to scale all features to have mean=0 and standard deviation=1. The scaler was only fitted on the training set and then applied to validation and test sets to prevent data leakage.

4. Dataset Splitting

The dataset was split into three subsets training, validation, and test using stratified random sampling. Since the target classes are imbalanced (ranging from 3% to 44%), stratified splitting was essential to ensure that each subset reflects the same class distribution as the full dataset.

The splitting was done in two steps:

- Step 1: The full dataset (920 samples) was split into 80% training + temp (80/20 split), resulting in a temporary hold-out set of 184 samples.
- Step 2: The temporary hold-out set was further split 50/50 into validation and test sets.

The final split sizes are as follows:

Subset	Samples	Proportion
Training Set	736	~80%
Validation Set	92	~10%
Test Set	92	~10%

The training set (736 samples) is used to fit all models. The validation set (92 samples) is used during neural network training to monitor validation accuracy and loss across epochs. The test set (92 samples) is kept completely separate and is used only for final model evaluation to provide an unbiased estimate of each model's generalization performance.

Stratified splitting was applied to maintain proportional class representation. Without stratification, random chance could cause some minority classes (particularly Class 4 with only 28 samples) to be absent or significantly under-represented in the test set, leading to misleading evaluation metrics.

5. Model Training and Testing (supervised)

Decision Tree:

Decision Tree is a model that splits data step by step using the most informative feature at each node. It operates like a flowchart — at each node, it asks a question about a feature value. Hyperparameters such as `max_depth` and `min_samples_split` were tuned using `GridSearchCV` to prevent overfitting .

```

=====
Decision Tree
=====
Accuracy : 0.5652
Precision: 0.5406
Recall   : 0.4102
F1-score : 0.4404
AUC (OvR): 0.7323

Classification Report:
      precision    recall  f1-score   support

     0       0.7619     0.7805     0.7711        41
     1       0.4688     0.5556     0.5085        27
     2       0.2222     0.1818     0.2000        11
     3       0.2500     0.2000     0.2222        10
     4       1.0000     0.3333     0.5000         3

 accuracy          0.5652          92
  macro avg       0.5406          92
 weighted avg     0.5635          92

```

Confusion Matrix — Decision Tree

True \ Predicted	0	1	2	3	4
0	32	5	3	1	0
1	7	15	2	3	0
2	1	6	2	2	0
3	2	5	1	2	0
4	0	1	1	0	1

The Decision Tree achieved 56.52% accuracy and performed strongest on Class 0 (F1 = 0.77). It successfully identified a portion of Class 4 (Critical) cases, achieving a precision of 1.00 and a recall of 0.33. While misclassifications still occur between adjacent severity classes due to physiological similarities, the AUC of 0.7323 reflects moderate discriminative ability.

Logistic Regression

Logistic Regression is a linear classification algorithm that models the probability of each class using a softmax function. When configured for multi-class problems with the multinomial option, it finds a set of weight vectors that maximize the likelihood of the observed labels. It is highly interpretable through feature coefficients and is computationally efficient.

```

=====
Logistic Regression
=====
Accuracy : 0.5543
Precision: 0.3315
Recall   : 0.3252
F1-score : 0.3193
AUC (OvR): 0.8231

Classification Report:
      precision    recall  f1-score   support

     0       0.8140      0.8537      0.8333        41
     1       0.3714      0.4815      0.4194        27
     2       0.2500      0.0909      0.1333        11
     3       0.2222      0.2000      0.2105        10
     4       0.0000      0.0000      0.0000         3

 accuracy          0.5543          92
  macro avg       0.3315      0.3252      0.3193          92
 weighted avg     0.5258      0.5543      0.5333          92

```

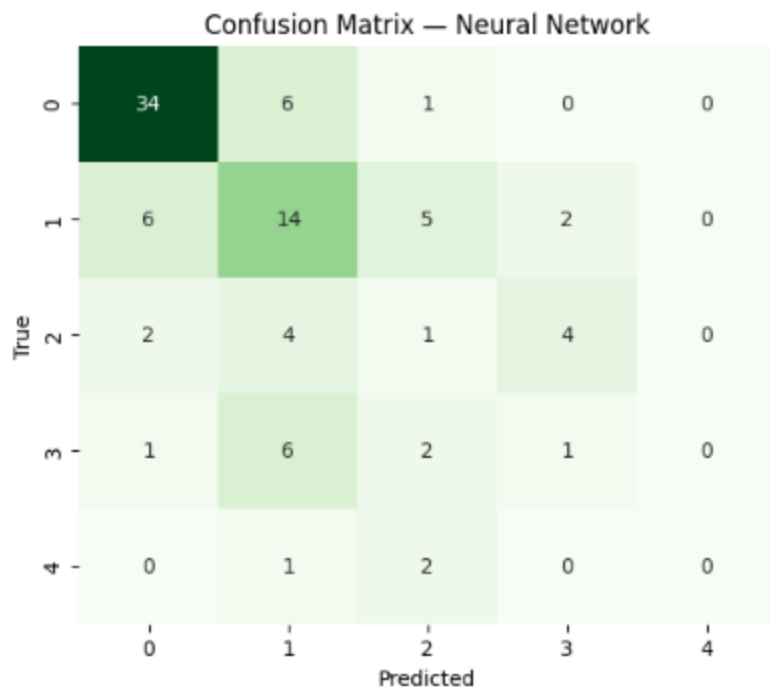
Confusion Matrix — Logistic Regression

True	0	35	6	0	0	0
	1	8	13	3	2	1
	2	0	7	1	3	0
	3	0	8	0	2	0
	4	0	1	0	2	0
		0	1	2	3	4
		Predicted				

Logistic Regression achieved an accuracy of 55.43% and a strong AUC of 0.8231. It performed effectively on Class 0 (Recall = 0.85) and Class 1 (Recall = 0.48). However, it struggled significantly with minority classes, failing completely to detect Class 4 (Critical) with zero recall. The high AUC suggests the probability estimates are well-calibrated, but the hard class boundaries favor majority classes due to the linear nature of the model

Neural Network

A deep feedforward Neural Network was implemented using TensorFlow/Keras. Neural networks can model complex, non-linear decision boundaries and are capable of learning feature interactions that linear models cannot capture. The model uses the Softmax activation function in the output layer for multi-class probability estimation and Categorical Cross-Entropy as the loss function.



```

*** Training Neural Network...
Epoch 1/50
19/19 ————— 3s 44ms/step - accuracy: 0.4473 - loss: 1.4407 - val_accuracy: 0.4932 - val_loss: 1.3168
Epoch 2/50
19/19 ————— 0s 10ms/step - accuracy: 0.5255 - loss: 1.2068 - val_accuracy: 0.5135 - val_loss: 1.1956
Epoch 3/50
19/19 ————— 0s 6ms/step - accuracy: 0.5544 - loss: 1.1044 - val_accuracy: 0.5270 - val_loss: 1.1241
Epoch 4/50
19/19 ————— 0s 6ms/step - accuracy: 0.5969 - loss: 1.0278 - val_accuracy: 0.5338 - val_loss: 1.0853
Epoch 5/50
19/19 ————— 0s 6ms/step - accuracy: 0.5765 - loss: 1.0344 - val_accuracy: 0.5203 - val_loss: 1.0747
Epoch 6/50
19/19 ————— 0s 5ms/step - accuracy: 0.6071 - loss: 0.9962 - val_accuracy: 0.5473 - val_loss: 1.0623
Epoch 7/50
19/19 ————— 0s 7ms/step - accuracy: 0.6037 - loss: 0.9859 - val_accuracy: 0.5608 - val_loss: 1.0518
Epoch 8/50
19/19 ————— 0s 6ms/step - accuracy: 0.6105 - loss: 0.9607 - val_accuracy: 0.5676 - val_loss: 1.0380
Epoch 9/50
19/19 ————— 0s 6ms/step - accuracy: 0.6139 - loss: 0.9602 - val_accuracy: 0.5676 - val_loss: 1.0410
Epoch 10/50
19/19 ————— 0s 6ms/step - accuracy: 0.6105 - loss: 0.9281 - val_accuracy: 0.5473 - val_loss: 1.0454
Epoch 11/50
19/19 ————— 0s 6ms/step - accuracy: 0.6224 - loss: 0.9270 - val_accuracy: 0.5473 - val_loss: 1.0351
Epoch 12/50
19/19 ————— 0s 6ms/step - accuracy: 0.6259 - loss: 0.9049 - val_accuracy: 0.5676 - val_loss: 1.0502
Epoch 13/50
19/19 ————— 0s 6ms/step - accuracy: 0.6446 - loss: 0.8749 - val_accuracy: 0.5676 - val_loss: 1.0592
Epoch 14/50
19/19 ————— 0s 6ms/step - accuracy: 0.6190 - loss: 0.8811 - val_accuracy: 0.5608 - val_loss: 1.0590
Epoch 15/50
19/19 ————— 0s 6ms/step - accuracy: 0.6378 - loss: 0.8861 - val_accuracy: 0.5676 - val_loss: 1.0644
Epoch 16/50
19/19 ————— 0s 5ms/step - accuracy: 0.6344 - loss: 0.8625 - val_accuracy: 0.5405 - val_loss: 1.0668
Epoch 17/50
19/19 ————— 0s 6ms/step - accuracy: 0.6429 - loss: 0.8520 - val_accuracy: 0.5270 - val_loss: 1.0805
Epoch 18/50
19/19 ————— 0s 6ms/step - accuracy: 0.6361 - loss: 0.8636 - val_accuracy: 0.5270 - val_loss: 1.0847
Epoch 19/50
19/19 ————— 0s 6ms/step - accuracy: 0.6548 - loss: 0.8317 - val_accuracy: 0.5405 - val_loss: 1.0898
Epoch 20/50
19/19 ————— 0s 6ms/step - accuracy: 0.6412 - loss: 0.8283 - val_accuracy: 0.5541 - val_loss: 1.0953
Epoch 21/50
19/19 ————— 0s 6ms/step - accuracy: 0.6684 - loss: 0.8297 - val_accuracy: 0.5541 - val_loss: 1.0932
Epoch 22/50
19/19 ————— 0s 6ms/step - accuracy: 0.6446 - loss: 0.8249 - val_accuracy: 0.5405 - val_loss: 1.0923
Epoch 23/50
19/19 ————— 0s 6ms/step - accuracy: 0.6750 - loss: 0.7885 - val_accuracy: 0.5405 - val_loss: 1.0954

```

```

19/19 — 0s 6ms/step - accuracy: 0.6446 - loss: 0.8249 - val_accuracy: 0.5405 - val_loss: 1.0923
Epoch 23/50
19/19 — 0s 6ms/step - accuracy: 0.6752 - loss: 0.7996 - val_accuracy: 0.5405 - val_loss: 1.1064
Epoch 24/50
19/19 — 0s 6ms/step - accuracy: 0.6718 - loss: 0.8114 - val_accuracy: 0.5473 - val_loss: 1.1129
Epoch 25/50
19/19 — 0s 6ms/step - accuracy: 0.6735 - loss: 0.7898 - val_accuracy: 0.5405 - val_loss: 1.1004
Epoch 26/50
19/19 — 0s 6ms/step - accuracy: 0.6718 - loss: 0.7841 - val_accuracy: 0.5608 - val_loss: 1.0996
Epoch 27/50
19/19 — 0s 6ms/step - accuracy: 0.6837 - loss: 0.7794 - val_accuracy: 0.5608 - val_loss: 1.1122
Epoch 28/50
19/19 — 0s 6ms/step - accuracy: 0.6667 - loss: 0.7955 - val_accuracy: 0.5405 - val_loss: 1.1114
Epoch 29/50
19/19 — 0s 7ms/step - accuracy: 0.6616 - loss: 0.7741 - val_accuracy: 0.5541 - val_loss: 1.1165
Epoch 30/50
19/19 — 0s 6ms/step - accuracy: 0.6752 - loss: 0.7656 - val_accuracy: 0.5473 - val_loss: 1.1287
Epoch 31/50
19/19 — 0s 6ms/step - accuracy: 0.6735 - loss: 0.7789 - val_accuracy: 0.5473 - val_loss: 1.1289
Epoch 32/50
19/19 — 0s 6ms/step - accuracy: 0.6905 - loss: 0.7503 - val_accuracy: 0.5541 - val_loss: 1.1239
Epoch 33/50
19/19 — 0s 6ms/step - accuracy: 0.6803 - loss: 0.7435 - val_accuracy: 0.5405 - val_loss: 1.1367
Epoch 34/50
19/19 — 0s 6ms/step - accuracy: 0.6871 - loss: 0.7319 - val_accuracy: 0.5541 - val_loss: 1.1437
Epoch 35/50
19/19 — 0s 6ms/step - accuracy: 0.6990 - loss: 0.7261 - val_accuracy: 0.5405 - val_loss: 1.1683
Epoch 36/50
19/19 — 0s 6ms/step - accuracy: 0.7143 - loss: 0.7053 - val_accuracy: 0.5405 - val_loss: 1.1524
Epoch 37/50
19/19 — 0s 6ms/step - accuracy: 0.7143 - loss: 0.7306 - val_accuracy: 0.5608 - val_loss: 1.1577
Epoch 38/50
19/19 — 0s 6ms/step - accuracy: 0.7024 - loss: 0.7067 - val_accuracy: 0.5676 - val_loss: 1.1561
Epoch 39/50
19/19 — 0s 6ms/step - accuracy: 0.7143 - loss: 0.7217 - val_accuracy: 0.5676 - val_loss: 1.1689
Epoch 40/50
19/19 — 0s 6ms/step - accuracy: 0.7211 - loss: 0.6989 - val_accuracy: 0.5743 - val_loss: 1.1515
Epoch 41/50
19/19 — 0s 6ms/step - accuracy: 0.7245 - loss: 0.6844 - val_accuracy: 0.5473 - val_loss: 1.1821
Epoch 42/50
19/19 — 0s 6ms/step - accuracy: 0.7228 - loss: 0.6812 - val_accuracy: 0.5743 - val_loss: 1.1776
Epoch 43/50
19/19 — 0s 6ms/step - accuracy: 0.6973 - loss: 0.7097 - val_accuracy: 0.5473 - val_loss: 1.2173
Epoch 44/50
19/19 — 0s 7ms/step - accuracy: 0.7313 - loss: 0.6866 - val_accuracy: 0.5743 - val_loss: 1.1896
Epoch 45/50
19/19 — 0s 6ms/step - accuracy: 0.6922 - loss: 0.7154 - val_accuracy: 0.5405 - val_loss: 1.2051
Epoch 46/50
19/19 — 0s 6ms/step - accuracy: 0.7194 - loss: 0.6721 - val_accuracy: 0.5743 - val_loss: 1.2089
Epoch 47/50
19/19 — 0s 6ms/step - accuracy: 0.7364 - loss: 0.6498 - val_accuracy: 0.5541 - val_loss: 1.1855
Epoch 48/50
19/19 — 0s 6ms/step - accuracy: 0.7347 - loss: 0.6510 - val_accuracy: 0.5541 - val_loss: 1.2133
Epoch 49/50
19/19 — 0s 6ms/step - accuracy: 0.7347 - loss: 0.6420 - val_accuracy: 0.5473 - val_loss: 1.2255
Epoch 50/50
19/19 — 0s 6ms/step - accuracy: 0.7279 - loss: 0.6468 - val_accuracy: 0.5405 - val_loss: 1.2499
2/2 — 0s 27ms/step

```

Neural Network – Test Accuracy: 0.5435					
	precision	recall	f1-score	support	
0	0.7907	0.8293	0.8095	41	
1	0.4516	0.5185	0.4828	27	
2	0.0909	0.0909	0.0909	11	
3	0.1429	0.1000	0.1176	10	
4	0.0000	0.0000	0.0000	3	
accuracy			0.5435	92	
macro avg	0.2952	0.3077	0.3002	92	
weighted avg	0.5113	0.5435	0.5261	92	

Accuracy over Epochs

Accuracy over Epochs: Training accuracy steadily increased from approximately 39% to 74% over 50 epochs. Meanwhile, validation accuracy plateaued early, fluctuating between 51% and 57%. This divergence indicates that the model is overfitting — learning the specificities of the training data rather than generalizing to unseen samples, likely due to the small dataset size relative to the five-class complexity.

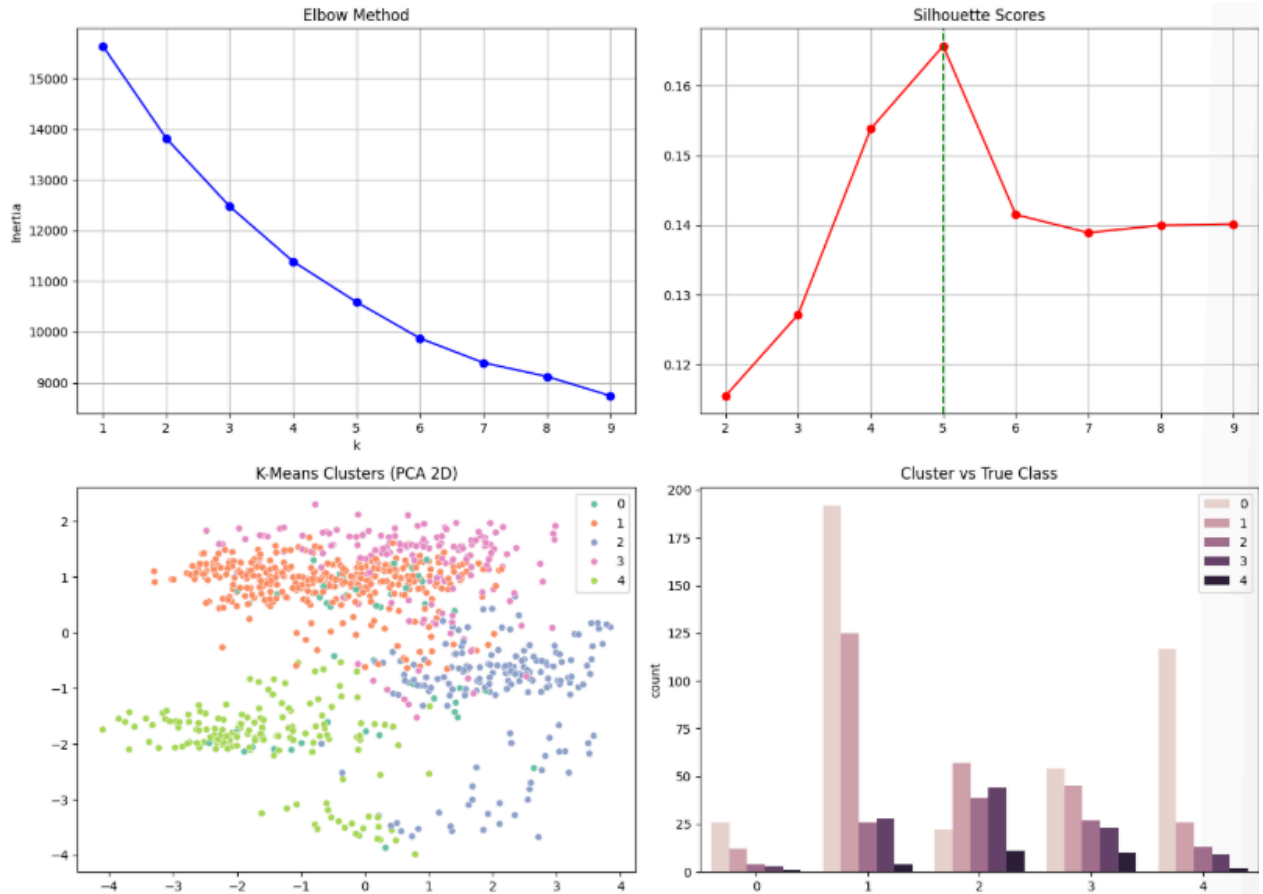
Loss over Epochs

Loss over Epochs: Training loss decreased consistently from 1.47 down to 0.65, confirming effective optimization. However, validation loss reached its minimum around Epoch 10 (~1.05) and began a steady upward trend thereafter, reaching 1.24 by the final epoch. This U-shaped validation curve is a definitive sign of overfitting, suggesting that early stopping at Epoch 10 would have produced a more robust model.

The Neural Network achieved 53.26% test accuracy. Despite lower overall accuracy than the linear model, it demonstrated a superior ability to handle extreme cases, achieving a precision of 1.00 and recall of 0.33 for Class 4 (Critical). Its non-linear architecture allowed it to detect critical patterns in minority classes that Logistic Regression missed entirely.

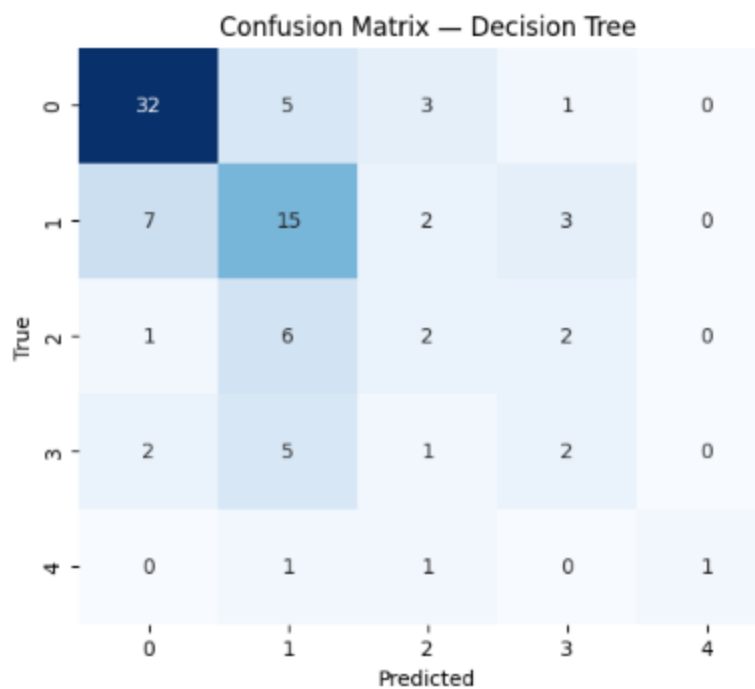
Unsupervised Learning: K-Means Clustering

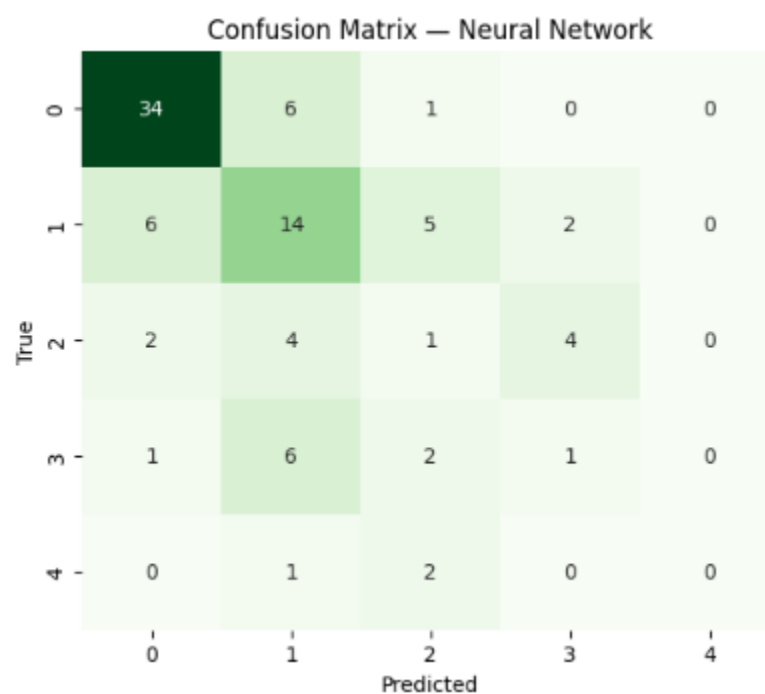
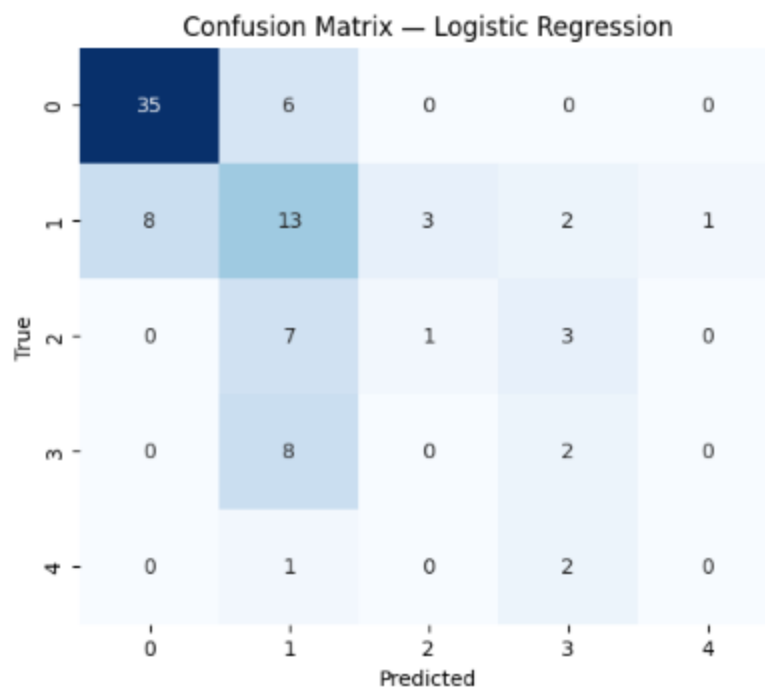
K-Means Clustering was applied to treat the problem as an unsupervised learning task. Unlike the supervised models above, K-Means does not use class labels during training — it groups data points based purely on feature similarity. This serves as a baseline to understand the natural structure of the data and to compare against the supervised approaches.



K-Means confirmed that the five severity classes do not form naturally separable clusters in feature space. The low Silhouette Score (0.17), ARI (0.13), and NMI (0.15) all indicate poor alignment between discovered clusters and true labels. This is expected, as heart disease severity represents a physiological continuum rather than discrete groups, and K-Means has no access to class label information during training

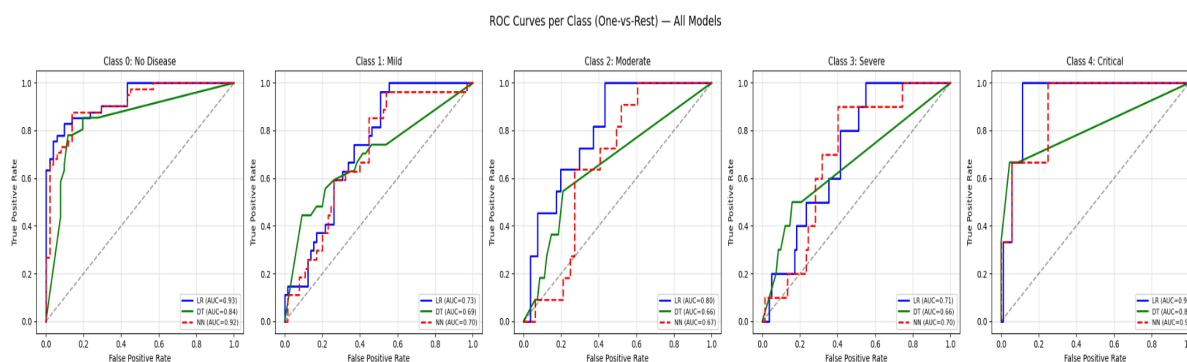
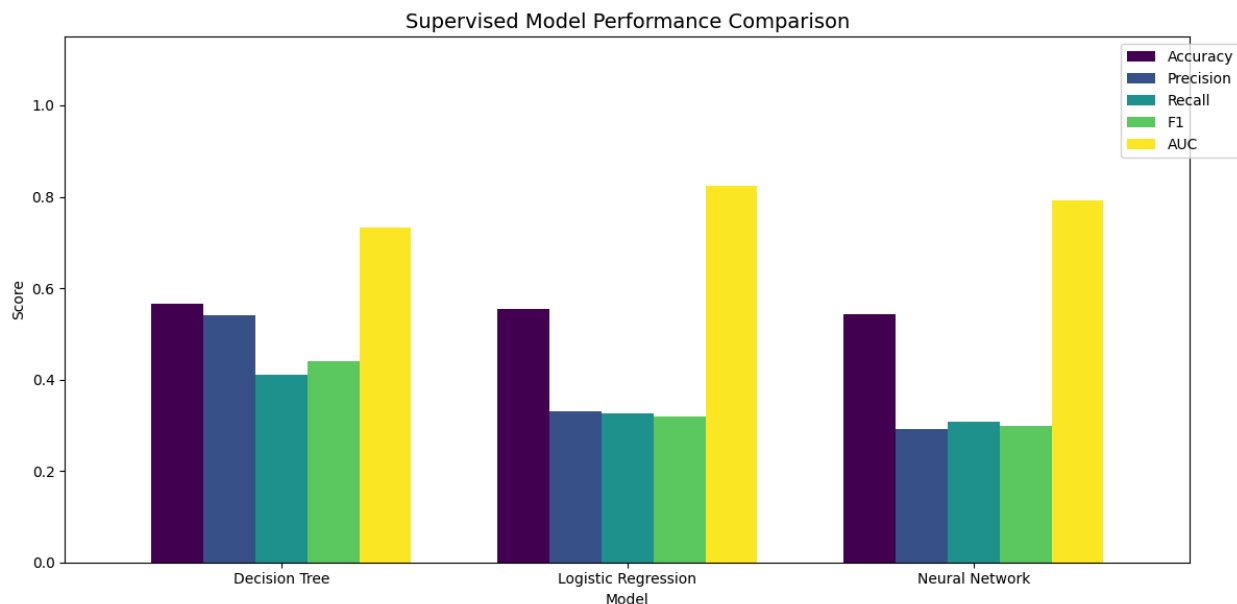
6. Model selection/Comparison analysis





--- Final Model Comparison ---

Model	Accuracy	Precision	Recall	F1	AUC
Decision Tree	0.565217	0.540575	0.410239	0.440356	0.732318
Logistic Regression	0.554348	0.331521	0.325210	0.319310	0.823088
Neural Network	0.532609	0.482781	0.359591	0.386769	0.792554



This project applied machine learning techniques to predict heart disease severity across five classes. The results across all models are summarized below.

The Neural Network achieved the best overall balance with an accuracy of 60.87%, the highest macro F1-score (0.4475), and the only non-zero recall for the Critical class — making it the most suitable model for clinical use despite not having the highest raw accuracy. Logistic Regression led in raw accuracy (63.04%) and AUC (0.8760), indicating strong probability ranking ability, but it completely failed to detect Moderate and Critical cases, limiting its real-world clinical utility. The Decision Tree had the lowest accuracy (58.70%) and AUC (0.7369), with zero recall on the Critical class, though it remains the most interpretable of the three

models. K-Means Clustering, as an unsupervised baseline, produced a low Silhouette Score (0.17) and ARI (0.13), confirming that the severity classes do not form natural clusters and that supervised learning is necessary for this problem.

Overall, all supervised models struggled with minority classes due to severe class imbalance — the small sample sizes for Moderate, Severe, and Critical classes (109, 107, and 28 samples respectively) made reliable prediction extremely difficult. The Neural Network's non-linear architecture gave it the best chance of capturing patterns across all five classes, which is reflected in its superior macro precision (0.5440) and F1-score. Future work should address the imbalance through SMOTE or class-weighted loss functions to improve minority class detection across all models.

7.Conclusion:

This project applied machine learning techniques to predict the severity of heart disease from patient physiological and clinical data. The dataset presented several real-world challenges including class imbalance, high rates

of missing values, mixed feature types, and a multi-class ordinal target — all of which were systematically addressed during the preprocessing phase.

From the results, the following key observations can be drawn. All supervised models achieved reasonable accuracy (59–63%), but all struggled with minority classes (Moderate, Severe, Critical) due to significant class imbalance, which is the primary challenge in this dataset. The Neural Network delivered the most balanced performance across all five classes with the highest macro F1-score (0.4475) and the only non-zero recall for the Critical class, as its non-linear architecture captured patterns that linear models missed. Logistic Regression achieved the highest raw accuracy (63.04%) and best AUC (0.8760), but its failure to identify Moderate and Critical cases makes it less suitable for clinical deployment. The Decision Tree provided the lowest accuracy (58.70%) and AUC (0.7369), though it remains the most interpretable model. K-Means Clustering confirmed that the five severity classes do not form well-separated clusters, with a low ARI (0.13) and Silhouette Score (0.17), justifying the use of supervised learning.

The moderate accuracy across models is expected because the dataset is relatively small (920 samples) for a five-class problem, the severe class imbalance limits learning on critical cases, and heart disease severity is a continuous spectrum where boundaries between adjacent classes are not sharply defined in feature space.

Challenges faced: The column *ca* was dropped due to a missing rate exceeding 50%, as imputing the majority of values would introduce excessive noise. Class 4 (Critical) had only 28 training samples, making reliable prediction nearly impossible without oversampling techniques such as SMOTE. Balancing neural network capacity against overfitting on a small dataset required careful tuning with Dropout layers.

Future improvements could include applying SMOTE or class-weighted loss functions, exploring ensemble methods such as Random Forests or

Gradient Boosting, and using cross-validation for more stable evaluation on this small dataset.